Java generics

What generics are, how classes, methods, bounds, and wildcards work, and why type erasure shapes all of it. The tone is how I would walk a teammate through each one, so it is plain and spoken; the design questions are fully how I would talk them out.

1. What are generics, why were they introduced, and what problems do they solve?

So generics let you **write a class or method that works over a type you fill in later**, like `List<String>` instead of a list of raw `Object`. Before Java 5 collections held `Object`, so you cast everything on the way out and there was nothing stopping you putting a `String` into a list you thought held `Integer`, you only found out at runtime with a `ClassCastException`. Generics solve exactly that: they move the type check to **compile time** and let you say "this list holds Strings, and the compiler will hold me to it." So the problem they fix is type-unsafe casting and the runtime blowups that came with it.

2. How do generics improve type safety and remove casting, and what happens without them?

Two sides of the same coin. **Type safety**: because you declared `List<String>`, the compiler rejects `list.add(42)` right there, so a whole class of bugs never reaches runtime. **No casting**: when you pull an element out, the compiler already knows it is a `String`, so you skip the `(String) list.get(0)` cast that raw code needed. **Without generics** you are back to raw `Object` collections: casts everywhere, and a wrong cast is a runtime `ClassCastException` instead of a compile error. So generics turn "hope the cast works" into "the compiler guarantees it."

3. What are the advantages and limitations of generics?

Advantages: compile-time type safety, no manual casts, and **reusable code** (one generic class works for any type, instead of one per type). **Limitations**, and these all trace back to type erasure (Q12): generics **do not exist at runtime**, so you cannot do `new T()`, `new T[]`, or `instanceof List<String>`; you **cannot use primitives** as type arguments (only reference types); and static members cannot use the class's type parameter. So they are a compile-time tool, and the limitations are the runtime blind spots.

4. What is a generic class, how do you create one, and where does the JDK use them?

A generic class **declares a type parameter in angle brackets** and uses it throughout: `class Box<T> { private T value; T get() {...} void set(T v){...} }`, and then `Box<String>` fixes `T` to `String`. You create it by putting `<T>` after the class name and using `T` as if it were a real type inside. The JDK is full of them: the **whole Collections framework** (`List<E>`, `Map<K,V>`, `Set<E>`), plus `Optional<T>`, `Comparable<T>`, and `Callable<V>`. Those are the everyday examples worth naming.

5. What can a generic class contain, and what are the gotchas?

Mostly what you expect, with a couple of sharp edges around `static`:

Question	Answer
Static methods	Yes, but they need their **own** type parameter; they cannot use the class's `T`
Static field of type `T`	**No**, a static field is shared across all instances, so it cannot use the per-instance `T`
Generic constructors	Yes, a constructor can declare its own type parameter
Extend another generic class	Yes (`class MyList<T> extends AbstractList<T>`)
Implement a generic interface	Yes (`class Box<T> implements Comparable<Box<T>>`)

The one people trip on is the static rule: `T` belongs to an instance, so anything `static` (shared across all instances) simply has no single `T` to refer to.

6. Type parameter v/s type argument, and the naming conventions.

Quick distinction: the **type parameter** is the placeholder in the declaration (the `T` in `class Box<T>`), and the **type argument** is the concrete type you pass when you use it (the `String` in `Box<String>`). The conventional single letters are just readability shorthand: **`T`** type, **`E`** element (collections), **`K`** / **`V`** key/value (maps), **`N`** number, **`R`** return type. Single letters are used because a type parameter is a placeholder, not a real domain name, so a short symbol keeps signatures readable, `Map<K,V>` reads better than `Map<KeyType,ValueType>` .

7. What is a generic method, how do you declare it, and how is it different from a generic class?

A generic method has **its own type parameter, declared before the return type**: `public <T> T firstOrNull(List<T> list) {...}` . The difference from a generic class is scope: a **generic class's** `T` is fixed when you create the object and shared by all its methods, while a **generic method's** `T` is scoped to that one call and inferred from the arguments each time you call it. And yes, a **non-generic class can absolutely contain generic methods**, the method just declares its own `<T>` . Utility classes like `Collections` are full of them.

8. What is a generic interface, and how do you implement it?

Same idea as a generic class, but a contract: `interface Repository<T> { void save(T item); T findById(String id); }` . You implement it either by **fixing the type** (`class UserRepository implements Repository<User>` , so `T` becomes `User` everywhere) or by **staying generic** (`class InMemoryRepo<T> implements Repository<T>` , passing the parameter through). `Comparable<T>` is the classic JDK example: `class Money implements Comparable<Money>` .

9. What are bounded type parameters, and watch out for `extends` v/s `super` .

A **bound restricts what the type parameter can be**. An **upper bound** with `extends` says "T must be this type or a subtype": `<T extends Number>` means T can be `Integer` , `Double` , and so on, and inside the method you can call `Number` 's methods on it. Here is the trap the question hints at: **`<T super Number>` is not legal Java**. A type parameter bound can only use `extends` (upper bound); `super` is only allowed on **wildcards**, not on type parameters. So there is no "lower bounded type parameter", lower bounds live entirely in wildcard land, which is the next question.

10. Wildcards: `<?>`, `<? extends T>`, `<? super T>`, and how a wildcard differs from `<T>`.

First the difference: `<T>` **names a type** you can refer to (you can use `T` inside), while `<?>` **is a wildcard**, an unknown type you cannot name, used when you only care about the shape, not the specific type. The three forms:

- **Unbounded** `<?>`: "a list of something, I do not care what." You can read it as `Object` and check size, but you cannot safely add to it. Good for read-only, type-agnostic code (`printAll(List<?> list)`).
- **Upper bounded** `<? extends T>`: "some subtype of T." You can **read** items as T, but you cannot add (the compiler does not know the exact subtype). This is a **producer**.
- **Lower bounded** `<? super T>`: "some supertype of T." You can **add** T into it, but reads come back as `Object`. This is a **consumer**.

11. What is PECS?

PECS is the mnemonic for choosing the right wildcard: **Producer Extends, Consumer Super**. If a structure **produces** values for you to read out, use `<? extends T>`; if it **consumes** values you put in, use `<? super T>`. The textbook example is a copy method: `copy(List<? super T> dest, List<? extends T> src)`, the source produces (extends), the destination consumes (super). Once PECS clicks, "which wildcard do I use" stops being a guess.

12. What is type erasure, why does Java use it, and how does it work?

This is the big one that explains most of generics' quirks. **Type erasure means generics are a compile-time-only feature**: the compiler uses the type information to check your code, and then **removes it**, so at runtime `List<String>` is just a raw `List`. Java did it this way for **backward compatibility**, generics arrived in Java 5 and the erased bytecode still had to run on and interoperate with pre-generics code and collections. How it works: the compiler replaces each type parameter with its **bound** (or `Object` if unbounded), and **inserts the casts** for you where you read values out.

13. What are the consequences of type erasure?

Everything awkward about generics comes from here. Because the type is gone at runtime:

- **Generics are not available at runtime**: you cannot do `instanceof List<String>` or `new T()`, because there is no `T` left to ask about.
- **You cannot create a generic array** (`new T[]`), because an array needs a real reified type at runtime; the usual workaround is an `Object[]` cast or `Array.newInstance`.

- **Primitives cannot be type arguments**: since everything erases toward `Object`, a type argument must be a **reference type**, so you use `Integer`, not `int` (autoboxing hides the cost).

14. What about generic varargs, nested generics, records, enums, and multiple type parameters?

Rapid-fire, because these come up as quick checks:

- **Multiple type parameters**: fine, just list them (`class Pair<A, B>`, `Map<K, V>`).
- **Nested generics**: also fine, and common (`Map<String, List<Order>>`).
- **Generic varargs**: allowed, but they cause the "unchecked" heap-pollution warning because varargs create an array (and arrays plus generics do not mix); you assert safety with `@SafeVarargs`.
- **Generic records**: yes, records can be generic (`record Pair<A, B>(A first, B second)`).
- **Generic enums**: no, an enum **cannot declare a type parameter**. It can still implement a generic interface, but the enum itself is not generic.

15. Do generics affect runtime performance?

No, and that surprises people. Because of erasure, generics are **purely a compile-time thing**, there is no reified type carried around, so a `List<String>` and a raw `List` run identically. So there is **no runtime overhead and no runtime speedup** from generics; the only cost is compile-time (type checking), which is negligible. The one indirect cost is that using wrappers like `Integer` instead of `int` (because primitives are not allowed) brings **boxing overhead**, but that is about primitives, not generics themselves.

16. You are designing a reusable library. How would generics improve the API?

The way I think about it: generics let me offer **one type-safe API that works for any caller's type**, instead of forcing everyone through `Object` and casts, or shipping a dozen near-identical classes. So a cache or a repository becomes `Cache<K, V>` or `Repository<T>`, and the caller gets **compile-time safety and no casting** for free, their `userRepo.findById(id)` returns a `User`, checked by the compiler. It also makes the API **self-documenting**: the signature says exactly what goes in and comes out. The discipline is to add type parameters where they genuinely buy safety and reuse, and not turn the signature into alphabet soup. *(Adapt to a library you have built.)*

17. Describe a production system where generics improved code quality.

The pattern I have leaned on is a **generic base for repositories or handlers**. Rather than every entity having its own copy-pasted CRUD class with casts, a `BaseRepository<T, ID>` held the shared operations type-safely, and each concrete repository just fixed the type. The win was concrete: **far less duplicated code**, and the compiler caught type mismatches that used to slip through as runtime cast errors. The same shape shows up with a generic result or response wrapper (`ApiResponse<T>`), which gave every endpoint a consistent, typed envelope. *(Adapt to your own example.)*

18. How would you teach a team to use generics without overengineering?

I would tell them generics are for **real reuse and real type safety, not for showing off the type system**. Use them when you are genuinely writing something that works across types (a container, a repository, a utility), and stop there. The overengineering smell is **too many type parameters** (`Thing<A, B, C, D>` that nobody can read) or wildcards sprinkled where a plain type would do. I would point them at **PECS** for the wildcard decision

so it is a rule rather than a guess, and tell them that if a signature has become hard to read, that is the signal to simplify, not to add another `<>`. Reuse justifies a type parameter; cleverness does not.

19. Common mistakes.

- **Using raw types** (`List` instead of `List<String>`), which throws away all the safety generics give you.
- **Using `Object` instead of generics**, so you are back to casting and runtime errors.
- **Misusing wildcards**, or **confusing `extends` and `super`**, usually solved by remembering PECS.
- **Ignoring unchecked-cast compiler warnings**, which are pointing at a real type hole.
- **Trying to create a generic array** (`new T[]`), which erasure does not allow.
- **Reaching for generics where plain polymorphism** (an interface or a base type) would be simpler.
- **Overcomplicating an API with too many type parameters**, so the signature is unreadable.
- **Misunderstanding PECS**, and picking the wrong wildcard for producer v/s consumer.